CI/CD Pipeline for Python Energy Management Application using Docker

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	**MAHAK SHRIMAL**	**EN22CS301566**
02	**MAHAK TAOSE**	**EN22CS301567**
03	**NAINA MANGHANI**	**EN22CS301626**
04	**MOHIT PATEL**	**EN22CS301610**
05	**PAAVAN PATNI**	**EN22CS301671**

Group Name:02D6

Project Number:DO-13

Industry Mentor Name:

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

TABLE OF CONTENTS

S.NO.	TOPICS	
1	INTRODUCTION	1.1 SCOPE OF THE DOCUMENT
		1.2 INTENDED AUDIENCE
		1.3 SYSTEM OVERVIEW
2	SYSTEM DESIGN	2.1 APPLICATION DESIGN
		2.2 PROCESS FLOW
		2.3 INFORMATION FLOW
		2.4 COMPONENTS DESIGN
		2.5 KEY DESIGN CONSIDERATIONS
		2.6 API CATALOGUE
3.	DATA DESIGN	3.1 DATA MODEL
		3.2 DATA ACCESS MECHANISM
		3.3 DATA RETENTION POLICIES
		3.4 DATA MIGRATION
4.	INTERFACES	
5.	STATE AND SESSION MANAGEMENT	
6.	CACHING	
7.	FUNCTIONAL REQUIREMENTS	7.1 AUTOMATED BUILD AND TESTING PROCESS
		7.2 APPLICATION CONTAINERIZATION USING DOCKER
		7.3 DEPLOYMENT ON AWS EC2
		7.4 MONITORING AND LOGGING
8.	NON-FUNCTIONAL REQUIREMENTS	8.1 SECURITY

		8.2 PERFORMANCE
		8.3 SCALABILITY
		8.4 RELIABILITY
9.	**REFERENCES**	

1. INTRODUCTION

The rapid growth of modern software systems requires efficient and automated processes for development, testing, and deployment. DevOps practices, particularly Continuous Integration and Continuous Deployment (CI/CD), play a crucial role in achieving faster delivery, improved reliability, and reduced manual effort. This project focuses on designing a CI/CD pipeline for an Energy Management Application that automates the complete software delivery lifecycle from code integration to deployment.The system integrates multiple DevOps tools including Git for version control, Jenkins or GitHub Actions for pipeline automation, Docker for containerization, and Amazon Web Services (AWS) for cloud-based deployment. Developers push code changes to a GitHub repository, which triggers the CI/CD pipeline to build, test, and package the application into a Docker container. The containerized application is then deployed on AWS EC2 instances, ensuring a scalable and consistent runtime environment.The primary objective of this system is to provide an automated, reliable, and efficient deployment process that minimizes human intervention and ensures consistent application behavior across different environments. By leveraging cloud infrastructure and containerization, the system enhances scalability, portability, and performance of the Energy Management Application.This High-Level Design document outlines the overall architecture and workflow of the system, providing a clear understanding of how different components interact to achieve continuous integration and deployment.

1.1 SCOPE OF THE DOCUMENT

This document presents the High-Level Design (HLD) of the CI/CD pipeline for the Energy Management Application. It defines the overall system architecture and explains how various DevOps tools such as Git, Docker, Jenkins or GitHub Actions, and AWS are integrated to automate the software development lifecycle.The document focuses on providing a clear understanding of system structure,workflow, and deployment strategy without going into low-level implementation details.

1.2 INTENDED AUDIENCE

1. Project Evaluators use this document to assess system design and feasibility.
2. DevOps Engineers use it to understand the CI/CD workflow and automation process.
3. Developers refer to it to understand how their code integrates into the pipeline.
4. System Administrators use it to manage deployment and infrastructure.

1.3 SYSTEM OVERVIEW

The system automates the complete software delivery lifecycle of the Energy Management Application. Developers push the application code to a GitHub repository, which triggers the CI/CD pipeline.The pipeline performs build, testing, and containerization using Docker, and finally deploys the application to AWS.This automated process ensures faster delivery, consistency across environments, and improved reliability of the system.

2. SYSTEM DESIGN

The system design of the CI/CD pipeline for the Energy Management Application is based on a modular and automated architecture that integrates multiple DevOps tools to ensure efficient software delivery. The application follows a containerized approach where the backend is developed using Python frameworks such as Flask or FastAPI, along with frontend technologies like HTML, CSS, and JavaScript. The application is packaged into Docker containers, which ensures consistent execution across different environments.

2.1 APPLICATION DESIGN

The Energy Management Application follows a containerized architecture where the backend is developed using Python frameworks such as Flask or FastAPI, along with frontend technologies like HTML, CSS, and JavaScript.The application is packaged into Docker containers, which ensures that it runs consistently across different environments.AWS is used to manage, scale, and maintain the availability of these containers. The system is divided into modules such as the Energy Monitoring Module, API Layer, CI/CD Pipeline Module, and Deployment Module.

2.2 PROCESS FLOW

The process begins when a developer writes code and pushes it to the GitHub repository. This action triggers the CI/CD pipeline configured using Jenkins or GitHub Actions. The pipeline pulls the latest code and performs the build process by installing dependencies and preparing the application. After the build, automated testing is executed to ensure code quality. If the tests pass successfully, a Docker image is created using a Dockerfile and pushed to a Docker registry.AWS then pulls this image and deploys it as containers within the cluster. Finally, the application becomes accessible to users through AWS services.

2.3 INFORMATION FLOW

1. Information starts from the developer's machine.
2. It moves to the Git repository.
3. The CI/CD pipeline processes the code.
4. Docker builds the application image.
5. The image is stored in the registry.
6. AWS deploys the application.
7. The final application is delivered to users.

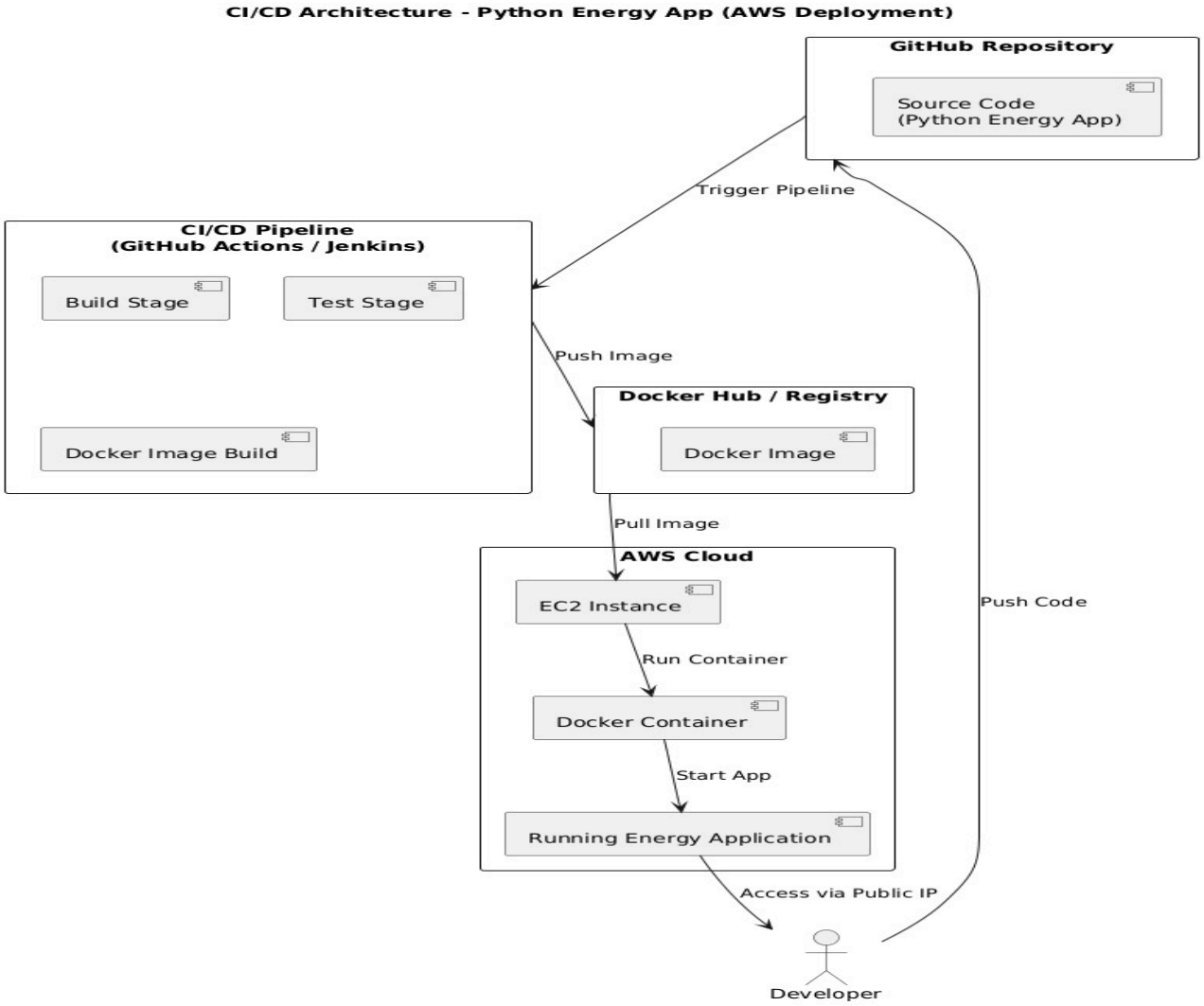

Figure 1 :High-Level System Architecture

2.4 COMPONENTS DESIGN

1. GitHub Repository stores the source code and triggers the pipeline.
2. CI/CD Pipeline automates build, testing, and deployment processes.
3. Docker containerizes the application and ensures consistency.
4. Docker Registry stores versioned Docker images.
5. AWS manages deployment, scaling, and availability.

These components work together to provide an automated and scalable system.

2.5 KEY DESIGN CONSIDERATIONS

1. Modularity ensures separation of components for better maintainability.
2. Scalability is achieved using Kubernetes auto-scaling features.
3. Reliability is maintained through automated testing and deployment.
4. Portability is ensured using Docker containers.

5. Automation reduces manual effort and human errors.

2.6 API CATALOGUE

1. AWS API is used by the CI/CD pipeline to manage deployments.
2. It helps in creating, updating, and scaling pods and services.

3. DATA DESIGN

The data design of the CI/CD pipeline for the Energy Management Application focuses on managing and tracking information across different stages of the software delivery lifecycle. The system maintains structured data related to source code repositories, pipeline executions, Docker images, and deployment instances on AWS. This data helps in ensuring traceability, monitoring, and efficient management of the application from development to deployment.

3.1 DATA MODEL

1. The system includes entities such as Repository, Pipeline Job, Docker Image, and Deployment Instance.
2. These entities represent different stages of the CI/CD pipeline.

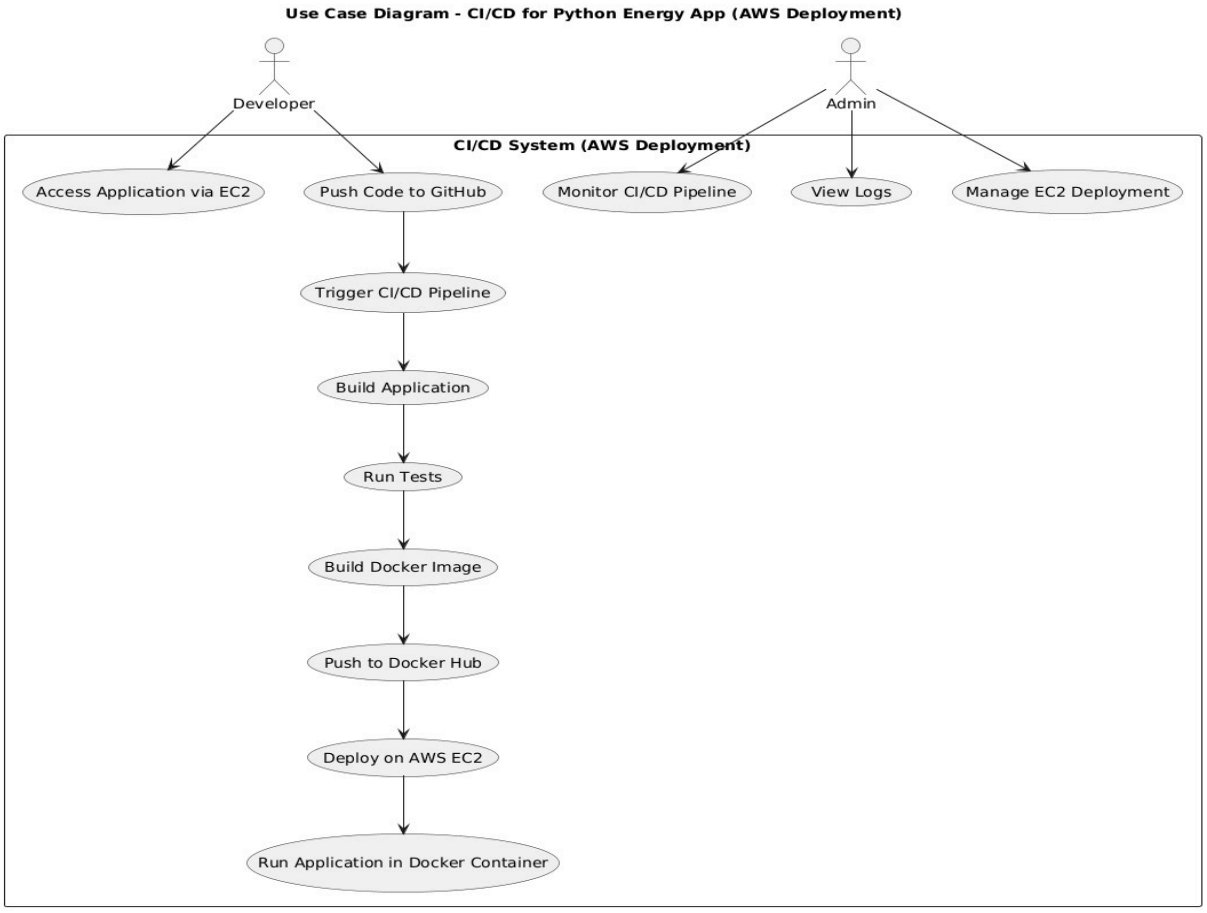

Figure 2 :Use Case Diagram

Logical Relationships (The "Lines")

1. **Actors in the System**

A. **Developer-**The Developer is responsible for writing code and pushing it to GitHub.The developer also accesses the deployed application through AWS EC2.
B. **Admin-**The Admin is responsible for monitoring the CI/CD pipeline, viewing logs, and managing deployment on AWS EC2.

2. Developer Interactions

A. The Developer pushes code to the GitHub repository.
B. This action automatically triggers the CI/CD pipeline.
C. The pipeline performs multiple stages including:Building the application,Running tests,Creating a Docker image,Pushing the image to Docker Hub
D. The application is then deployed on AWS EC2.
E. Finally, the Developer can access the running application from the EC2 instance.

3. CI/CD System Workflow

A. The system starts when code is pushed to GitHub.
B. The CI/CD pipeline is triggered automatically.
C. The application is built and tested.
D. A Docker image is created and stored in Docker Hub.
E. AWS EC2 pulls the Docker image and deploys it.
F. The application runs inside a Docker container.

4. Admin Interactions

A. The Admin monitors the CI/CD pipeline to check its status.
B. The Admin views logs generated during build and deployment.
C. The Admin manages AWS EC2 deployment, such as starting, stopping, or updating instances.

3.2 DATA ACCESS MECHANISM

1. GitHub access is controlled using authentication tokens.
2. Docker registry requires secure credentials for access.
3. AWS resources are accessed using IAM roles and permissions.

3.3 DATA RETENTION POLICIES

1. Pipeline logs are stored for monitoring and debugging.
2. Docker images are versioned for rollback.
3. AWS maintains deployment logs and instance data.

3.4 DATA MIGRATION

1. New application versions create new Docker images.
2. AWS deploys updated containers on EC2 instances.
3. Older versions can be restored if needed.

4. INTERFACES

The system provides multiple interfaces that enable interaction between users and the CI/CD pipeline as well as communication between different system components. These interfaces ensure smooth operation, monitoring, and management of the Energy Management Application throughout its lifecycle.The primary interfaces in the system include the user interface and the command-line interface.

4.1 USER INTERFACE

1. GitHub interface is used for code management.
2. Jenkins/GitHub Actions dashboard is used to monitor pipelines.
3. AWS Management Console is used to manage deployment.

These interfaces provide visibility and control over the system.

4.2 COMMAND LINE INTERFACE

1. Git CLI is used for version control operations.
2. Docker CLI is used for building and running containers.
3. AWS CLI is used for managing cloud resources.

CLI tools provide flexibility for developers and engineers.

5. STATE AND SESSION MANAGEMENT

1. AWS EC2 maintains the running state of the application.
2. CI/CD pipeline tracks build and deployment stages.
3. Application sessions are managed using secure mechanisms.

This ensures system stability and availability.

6. CACHING

1. Docker layer caching improves build performance.
2. Dependency caching reduces installation time.
3. Pipeline artifacts are reused to optimize efficiency.

Caching improves system speed and performance.

7. FUNCTIONAL REQUIREMENTS

The functional requirements define the core functionalities that the CI/CD system must provide to ensure automated development, testing, containerization, and deployment of the Energy Management Application. These requirements describe how the system behaves and the services it offers to users such as developers and administrators.

7.1 AUTOMATED BUILD AND TESTING PROCESS

1. The system must automatically build the application whenever new code is pushed to the repository.
2. It should install dependencies and prepare the application for execution.
3. The system must run automated tests such as unit testing and integration testing.

7.2 APPLICATION CONTAINERIZATION USING DOCKER

1. The system must package the application into Docker containers.
2. A Dockerfile should define the runtime environment and dependencies.
3. The containerized application should run consistently across development, testing, and production environments.

7.3 DEPLOYMENT ON AWS EC2

1. The system must deploy the containerized application on AWS EC2 instances.
2. The EC2 instance should pull the Docker image from a Docker registry.
3. The application must run inside a Docker container on the EC2 server.

7.4 MONITORING AND LOGGING

1. The system must provide monitoring of the CI/CD pipeline execution.
2. It should generate logs for each stage such as build, test, and deployment.
3. Logs must be accessible to administrators for debugging and analysis.

8. NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements describe the quality attributes and constraints of the CI/CD system for the Energy Management Application. These requirements ensure that the system performs efficiently, remains secure, and scales effectively while maintaining reliability and usability.

8.1 SECURITY

1. Secure authentication for GitHub and AWS access.
2. Use of IAM roles for AWS security.
3. Protection of sensitive data using environment variables.

8.2 PERFORMANCE

1. Fast automated deployment process.
2. Efficient resource utilization on AWS.
3. Reduced manual effort.

8.3 SCALABILITY

1. AWS allows scaling of EC2 instances based on demand.
2. The system can handle increased workload efficiently.

8.4 RELIABILITY

1. Continuous monitoring of applications on AWS.
2. Automatic restart of services if failure occurs.
3. Consistent deployment using CI/CD pipeline.

9. REFERENCES

[1] DevOps Course Material, *Medicaps University – Datagami Skill Based Course*, 2025–2026.

[2] Git Documentation, "Git Official Documentation," [Online]. Available: https://git-scm.com/docs

[3] Docker Documentation, "Docker Official Documentation," [Online]. Available: https://docs.docker.com

[4] Amazon Web Services Documentation, "AWS EC2 Documentation," [Online]. Available: https://docs.aws.amazon.com/ec2

[5] Jenkins Documentation, "Jenkins User Documentation," [Online]. Available: https://www.jenkins.io/doc

[6] GitHub Actions Documentation, "GitHub Actions Documentation," [Online]. Available: https://docs.github.com/actions

[7] Docker Hub Documentation, "Docker Hub Documentation," [Online]. Available: https://hub.docker.com

[8] Energy Management Application Project Repository, GitHub, 2026.

10. GITHUB

https://github.com/Mohit150105/Energy-App